

Basic (guess token counts, then check) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried guessing first. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (a real estimation method), **Advanced** (why the rule works the way it does, and where that starts to matter for money). Read Basic first even if you already feel confident — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and a worked example](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Don't count words. Count roughly how many groups of 4 letters the sentence has — that's closer to how tokens actually work.

A short, plain sentence with normal English words will usually land close to "word count $\times$ 1.3" tokens (a little more tokens than words, not fewer), because some words split into two pieces.

Things to use:

- Character count $\div$ 4 — one estimate.
- Word count $\times$ 1.3 — a second, independent estimate.
- platform.openai.com/tokenizer — where you check the real answer.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The rule of thumb from Core Concepts is: 1 token $\approx$ 4 characters $\approx$ $\frac{3}{4}$ of a word, for plain English. To estimate a sentence, count its characters (including spaces) and divide by 4 — or count its words and multiply by roughly 1.3.

The two estimates won't always agree exactly, and that's fine — the goal isn't a precise number, it's being close enough to make a reasonable cost or context-limit decision without running code. Watch especially for: long or unusual words (these often split into 2+ tokens), and punctuation (commas and periods are usually their own token, adding a small amount you might forget to count).

The exact pieces:

- **chars $\div$ 4** — fastest to compute in your head, works fine on plain English prose.
- **words $\times$ 1.3** — the alternative estimate; the two rarely land on exactly the same number, which is itself worth noticing.
- **Choose 5 sentences that deliberately span a range** — a very short one, a normal one, a long one with uncommon words, one with numbers or punctuation, and one that's mostly simple, common words.

Write down your character-based estimate and your word-based estimate separately for each sentence, before checking the real tool.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Both estimates are describing the *average* behavior of a tokenizer built by finding the most common recurring chunks in a mostly-English training set — which means the rule of thumb isn't equally wrong in both directions. It tends to *undercount* rare or long words (they split into more pieces than "1 word" suggests) and it tends to be *roughly right* on short, common, everyday words. Knowing which direction the error usually runs in is more useful than knowing the average error size, because it tells you when to pad your estimate upward on purpose rather than trust the number as-is.

There's a production-grade reason to care about getting this right beyond "will my prompt fit": **prompt caching** (previewed here, covered properly in Doc12) only reuses a cached prefix if the tokens match *exactly* — not the words, the tokens. Two prompts that look identical to a human but differ by one trailing space, one extra blank line, or a slightly different phrasing of the same instruction can tokenize differently and silently miss the cache, paying full price on every call instead of the cached, discounted rate. A rough "close enough" token estimate is fine for a one-off cost check; it is not enough to reason about whether a system prompt sent thousands of times a day is actually hitting the cache the way you assumed it was.

The real question worth carrying forward: for a prompt you send *once*, $\text{chars} \div 4$ is plenty. For a prompt (like a system prompt) you send on every single call of a running feature, the exact token count — and whether it's byte-for-byte identical between calls — is worth actually measuring with `tiktoken`, not estimating.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both give you two independent ways to *estimate* a token count for a one-off check. Advanced explains *which direction* the rule of thumb tends to be wrong in (it undercounts rare/long words, and is roughly accurate on common short words), and raises a case where "close enough" genuinely isn't good enough — a prompt resent on every call, where exact token-level identity (not just approximate count) determines whether prompt caching actually saves you money.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and a worked example

Basic Version

```
for each of your 5 sentences:  
    count the words  
    guess: words × 1.3, rounded  
    write the guess down  
  
then, one at a time:  
    paste the sentence into the tokenizer tool  
    write down the real number  
    compare — how far off were you?
```

A worked example: "The cat sat on the mat." → 6 words → guess: about 7-8 tokens. Real answer (check it yourself): close to that, because it's all short, common words.

Now do the same process for your other 4 sentences — pick ones that are progressively less "easy" (longer, less common words, more punctuation) — before checking the Intermediate Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
sentences = [a short one, a normal one, a long/uncommon-word one, a punctuation/n

for each sentence:
    char_guess = round(len(sentence) / 4)
    word_guess = round(word_count(sentence) * 1.3)
    write both down

for each sentence, in the tokenizer tool:
    actual = real token count
    error_chars = actual - char_guess
    error_words = actual - word_guess

compare: which estimate method was closer, and on which kind of sentence?
```

A worked example, using both estimate methods: "The cat sat on the mat." → 26 characters (including spaces) → char-based guess: $26 \div 4 \approx 7$. Word-based guess: 6 words $\times 1.3 \approx 8$. Both estimates land close together here, because this sentence is exactly the case the rule of thumb was built for — short, common, plain English words.

Now pick 4 more sentences that each break one assumption on purpose (uncommon words, heavy punctuation, a very short fragment, a longer paragraph) and repeat both estimates for each, before checking the Advanced Version below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

for a system-prompt-shaped piece of text (something you'd actually resend every call, not a one-off question):

estimate its token count using both methods, same as above
then ask: if I paste this exact same text in twice, with one trailing space added the second time, would you expect the token sequence to still match exactly? guess yes or no, and why, before checking.

A worked example of the caching-relevant question: take a short system prompt like "You are a helpful assistant. Answer concisely." and compare its token count against the same string with a single trailing space added: "You are a helpful assistant. Answer concisely. ". Guess first — does the trailing space change the token count, the token *sequence*, both, or neither? Check both in the real tokenizer tool and note exactly what changed.

Write down what you found — including whether it surprised you — before checking the full [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both build and check a one-off token estimate for ordinary sentences. Advanced asks a sharper question about a prompt meant to be resent unchanged on every call: does a trivial, easy-to-miss difference (one trailing space) change its tokenization at all — which is exactly the kind of gap between "looks the same to me" and "is the same to the model" that determines whether prompt caching actually works the way you assumed.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)